



영상처리

4 주차 보고서

컴퓨터융합학부

202102699 정민경



1. 과제 설명

함수이름	my_bilinear()
코드설명	my_bilinear 실습코드를 참고해 코드를 구현하였다. 입력받은 src image의 shape에 비율을 곱해서 그 size만큼의 배열을 return 해 주었다. 이 배열을 채우는 값 수식은 이론시간에 배운 방법 그대로 수행하도록 구현 해주었고, 이 my_bilinear 함수를 통해 interpolation을 수행하도록 하였다.
이미지	<pre> 10 (h, w) = src.shape 11 h_dst = int(h * scale + 0.5) 12 w_dst = int(w * scale + 0.5) 13 14 dst = np.zeros((h_dst, w_dst), dtype=np.float32) 15 16 for row in range(h_dst): 17 for col in range(w_dst): 18 # 스케일링 되기 전 original 좌표 19 y = row / scale 20 x = col / scale 21 22 # bilinear interpolation 23 # 1. (y, x)를 기준으로 좌측위, 좌측아래, 우측아래, 우측위 좌표를 구함. 24 # 2. bilinear interpolation 식에 따라 해당 row, col 좌표에 값을 대입 25 y_up = int(y) # 버림 26 y_down = min(int(y + 1), h - 1) # 반올림 단 src의 최대 좌표값보다는 같거나 작게 27 x_left = int(x) # 버림 28 x_right = min(int(x + 1), w - 1) # 반올림 단 src의 최대 좌표값보다는 같거나 작게 29 30 t = y - y_up 31 s = x - x_left 32 33 intensity = ((1 - s) * (1 - t) * src[y_up, x_left]) \ 34 + (s * (1 - t) * src[y_up, x_right]) \ 35 + ((1 - s) * t * src[y_down, x_left]) \ 36 + (s * t * src[y_down, x_right]) 37 dst[row, col] = intensity 38 39 dst = np.round(dst).astype(np.uint8) 40 return dst </pre>

함수이름	my_upsampling_laplacian()
코드설명	이 함수에서는 기존 x에 residual을 더해서 반환하는 함수이다. Laplacian_residual의 모든 값을 x와 더해주었고, 값이 max 값인 255를 넘으면 안되므로, np.clip을 통해 0보다 작으면 0으로, 255보다 크면 255로 바꿔 반환해주었다.
이미지	<pre> 105 if residuals is not None: 106 x = x + laplacian_residuals[i] 107 108 x = np.clip(x, 0, 255) 109 x = np.round(x).astype(np.uint8) </pre>

함수이름	my_downsampling_pyramid() - laplacian pyramid
코드설명	gaussian filtering이 진행된 image를 받아서 downsampling 해주는 과정에서, np.floor 함수로 소숫점을 잘라주고, ratio, ratio 만큼 잘라서 downsampling을 해주었다. 이렇게 downsampling 한 image를 가지고 다시 upsampling을 진행하게 되는데 이 때 사용하는 함수가 my_upsampling_laplacian이다. 이때 src를 앞에서 구한 downsampling이미지로 넘겨주어 upsampling을 진행하도록 구현하였다. 그 후 Laplacian_residual을 구하기 위해 현재 level의 gaussian image에서 upsampling image를 제거해주어 laplacian을 구하였다.
이미지	<pre> 151 # 1. filtering 152 filtered_img = cv2.Filter2D(gaussian_pyramid[-1], -1, gaussian_filter) 153 154 # 2. residual 계산 및 저장 155 downsampling_img = np.floor(filtered_img[::ratio, ::ratio]) 156 upsampling_img = my_upsampling_laplacian(downsampling_img, ratio=2, residuals=None, upsampling_type=interpolation_type, 157 pyramid_level=1) 158 laplacian_residual = gaussian_pyramid[level] - upsampling_img 159 laplacian_residuals.append(laplacian_residual) 160 161 162 # 3. 2.1에서 downsampling한 이미지를 다음 피라미드의 입력으로 설정 163 gaussian_pyramid.append(downsampling_img) 164 165 return laplacian_residuals, gaussian_pyramid[-1] 166 </pre>

함수이름	my_downsampling_pyramid() - naïve pyramid
코드설명	바로 위의 downsampling_pyramid와는 다르게 이 함수는 단순히 비율만큼 잘라서 다음 level image로 줄이는 과정이다. 그렇기 때문에 위의 함수처럼 laplacian을 구하고자 upsampling을 하고, 제거하는 과정으로 줄이는 것이 아닌 [::ratio, ::ratio]로 잘라 append 시켜주었다.
이미지	<pre> if filter_size is None: for level in range(pyramid_lvl): ##### # TODO naïve pyramid # TODO subsample(즉 downsampling 이미지 크기 줄이기) # TODO nearest 와 bilinear 중 경우에 맞는 한 가지를 택해서 사용 # TODO interpolation_type 변수를 활용 ##### out_img = gaussian_pyramid[-1][::ratio, ::ratio] gaussian_pyramid.append(out_img) return gaussian_pyramid[-1] </pre>

2. 결과 이미지

Original	 A black and white photograph of a woman with long dark hair, wearing a light-colored straw hat with a wide brim and a decorative feather. She is looking over her shoulder towards the camera. The image is displayed in a window titled 'original_202102699'.
Nearest + Residual	 A black and white photograph of the same woman in the same hat, appearing to be a reconstructed or denoised version of the original image. It is displayed in a window titled 'nearest_202102699'.

Nearest No
Residual



Nearest
No
Residual
No
Gaussian
filter



**Bilinear +
Residual**



**Bilinear
No
Residual**



Bilinear
No
Residual
No
Gaussian
filter



실행 시 평가

```
/Users/jeongmingyeong/CNU/IP_23/venv/bin/python /  
원본과 nearest 보간법을 이용해서 복원한 경우 비교 결과: True  
원본과 bilinear 보간을 이용해서 복원한 경우 비교 결과: True
```

3. 이미지 분석

- Up sampling 과정에서 Interpolation 방법에 따른 차이

- Nearest :

- 이미지의 크기를 키울 때 가장 가까운 픽셀값을 사용하는 방식이다. 그렇기 때문에 nearest interpolation 방식으로 upsampling을 진행하면 부드럽지 않고 픽셀 각각이 튀는 느낌으로 upsampling이 진행되는 것을 볼 수있다.

- Bilinear :

- Bilinear interpolation 방식은 이미지의 크기를 키울 때 주변 픽셀값을 모두 고려해서 크기를 키우는 방식이다. 가까운 픽셀에 가중치를 더 주고, 먼 픽셀 역시 고려는 하지만 가중치를 조금 주어서 주변에 있는 픽셀을 고려해 upsampling을 진행했다. 그렇기 때문에 Nearest보다 upsampling된 사진이 좀 더 부드러운 것을 볼 수있다.

- Residual 을 적용하지 않았을 때 이미지가 어떻게 변하는지

- Residual 을 적용해 upsampling을 진행하면 original과 거의 비슷한 결과가 나오는 것을 볼 수 있다.

- 하지만 residual을 적용하지 않는다면, downsampling 된 사진이 resize되어 크기가 커지는 것이기 때문에 image가 선명하지 않고, 많이 깨지는 것을 볼 수있다.

- Residual 없이 upsampling 한 사진에서 gaussian filter까지 없다면 사진이 더 pixel 처럼 각져있는 결과를 얻을 수 있다.